



Escuela de Física

Ingeniería Física

Instructivo de Laboratorio

Instrumentación I

20 de mayo de 2026

Versión: 0.2

Realizado por: Juan J. Rojas y Eduardo Font

Índice general

1. Sensores de estímulos eléctricos	2
1.1. Objetivos	2
1.2. Materiales y equipo	2
1.3. Procedimiento	2
1.4. Preguntas para el análisis de resultados	3
2. Sensores de temperatura	5
2.1. Objetivos	5
2.2. Materiales y equipo	5
2.3. Procedimiento	5
2.4. Análisis	7
3. Sensores de presión	8
3.1. Objetivos	8
3.2. Materiales y equipo	8
3.3. Actividad 1	8
3.3.1. Procedimiento	8
3.3.2. Análisis	11
3.4. Actividad 2	11
3.4.1. Procedimiento	11
3.4.2. Análisis	11
4. Mediciones de nivel	12
4.1. Objetivos	12
4.2. Materiales y equipo	12
4.3. Actividad 1	12
4.3.1. Procedimiento	12
4.3.2. Análisis	14
4.4. Actividad 2	14
4.4.1. Procedimiento	14
4.4.2. Análisis	14
5. Sensores de peso	15
5.1. Objetivos	15
5.2. Materiales y equipo	15
5.3. Actividad 1	15
5.3.1. Procedimiento	15

5.3.2. Análisis 17

Laboratorio 1

Sensores de estímulos eléctricos

1.1. Objetivos

Al finalizar este laboratorio el estudiante estará en capacidad de:

- Programar un Arduino con funciones básicas como introducción al microcontrolador.
- Calcular el error asociado a la medición utilizando una multimetro digital de alta precisión.

1.2. Materiales y equipo

A suministrar por la Escuela:

- 1 ARDUINO UNO R4 MINIMA
- Cable USB de conexión del Arduino
- 1 potenciómetro (5 k Ω o más)
- 2 LEDs
- 2 resistencias de 330 Ω (o un valor cercano)
- Cables de interconexión (jumpers) macho-macho

1.3. Procedimiento

- a. Realice las conexiones del circuito tal y como se indica en la Figura 1.1.
- b. Abra el programa Arduino IDE

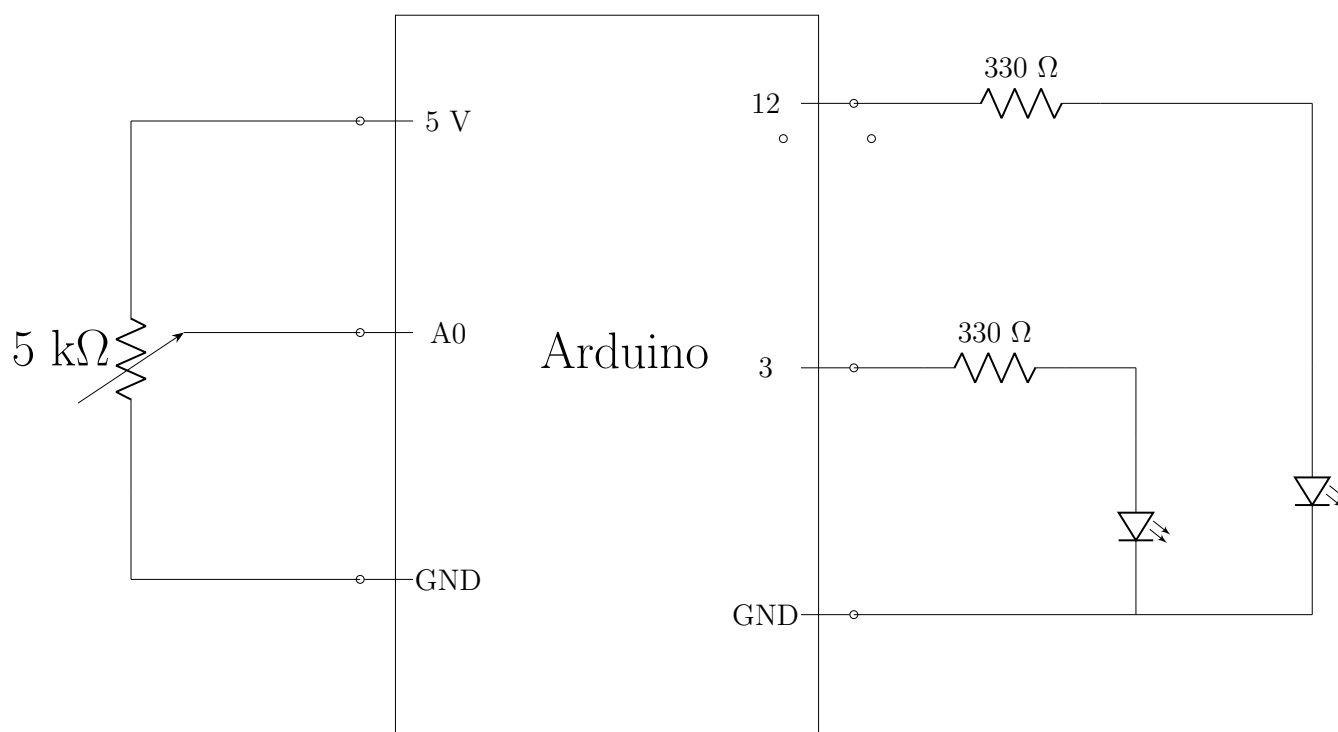


Figura 1.1: Circuito de Arduino con potenciómetro y dos LEDs.

- Añada en la función *loop()* del código una sección para leer el valor analógico del pin A0 e imprimirlo a la consola serial. Córralo y verifique que al variar el potenciómetro, varíe correctamente el valor.
- Modifique el *loop()* y agregue un delay, que varíe linealmente entre 100 ms y 1 s, dependiendo de la posición del potenciómetro. Córralo y verifique que ahora las lecturas se impriman en la terminal con diferentes tiempos de retraso, variando con el valor del potenciómetro.
- Agregue a la función *loop()* el código necesario para que el LED en el pin 12 se encienda y se apague con el retraso dado (para esto hay que agregar un segundo delay para controlar el tiempo de apagado). Córralo y verifique que los tiempos de encendido y apagado del LED varíen correctamente al variar el potenciómetro.
- Agregue el código necesario en *loop()* para que el brillo del LED varíe también con el valor leído del potenciómetro. Córralo y verifique que el brillo del LED varíe correctamente al cambiar el valor del potenciómetro.
- Mida la tensión entre la tierra y el pin A0 (el pin central del potenciómetro) con un multímetro y compare las lecturas obtenidas con el Arduino y el multímetro (al menos 5 veces).

1.4. Preguntas para el análisis de resultados

- ¿Cuál es el rango de valores que devuelve *analogRead()* en un Arduino UNO?
- Con el resultado anterior, ¿cuántos bits de resolución tiene el ADC del Arduino UNO?

- ¿Cómo se convierte de los valores obtenidos con *analogRead()* a una medición de tensión eléctrica?
- ¿Cómo calculó la fórmula para ajustar los valores obtenidos de *analogRead()* a valores entre 100 ms y 1 s para la función *delay()*?
- Calcule el error relativo y el error absoluto para cada una de las 5 lecturas de tensión entre el Arduino y el potenciómetro (asuma que el multímetro es el valor real).
- Incluya el código que realizó en esta práctica en el informe.

Laboratorio 2

Sensores de temperatura

2.1. Objetivos

Al finalizar este laboratorio el estudiante estará en capacidad de:

- Utilizar un microcontrolador para realizar mediciones de temperatura.
- Realizar un algoritmo básico para obtener mediciones e imprimirlas en la terminal.

2.2. Materiales y equipo

A suministrar por la Escuela:

- 1 Arduino UNO R4 MINIMA
- 1 termistor de 10 k Ω NTC marca TDK
- 1 breadboard
- Cables de conexión macho-macho

2.3. Procedimiento

- a. Arme el circuito tal como se muestra en la Figura 2.1.

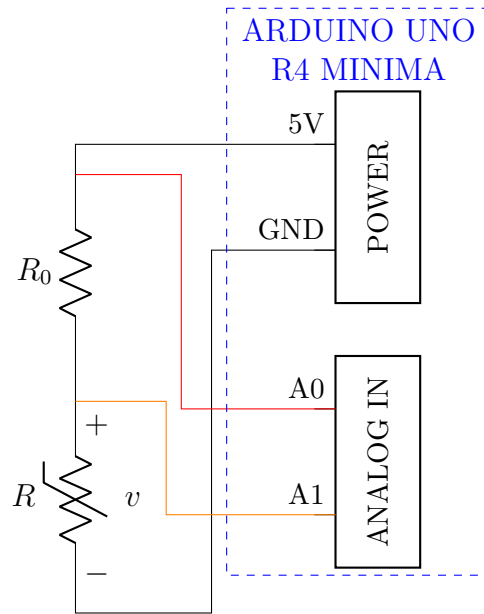


Figura 2.1: Conexión de circuito para el Laboratorio 3

- b. Usando la formula del divisor de tensión y simplificando se llega a que:

$$\frac{R_0}{R} = \frac{5V}{V} - 1$$

- c. El valor de la temperatura se puede aproximar usando la siguiente ecuación:

$$\frac{1}{T} = \frac{1}{T_R} + \frac{1}{B_{25/100}} \ln \left(\frac{R}{R_R} \right)$$

donde T es la temperatura del termistor en K, T_R es la temperatura de referencia en K y R_R es la resistencia de referencia en Ω

- d. Según la [hoja de datos](#) del termistor el valor de $B_{25/100}$ es igual a 4300.
- e. Implemente la ecuación en Arduino, leyendo V con la entrada analógica.
- f. Suponga que su sensor será utilizado para medir la temperatura de la piel (20°C a 40°C), para reducir el error en la medición utilice lo mostrado en la sección 2.3 de [este documento](#) para determinar C_0 , C_1 y C_3 .
- g. Realice una función en el mismo sketch en Arduino IDE que permita calcular el valor de T_2 a partir de los valores de C_0 , C_1 , C_3 y R usando la ecuación de Steinhart-Hart
- h. Imprima en la terminal los valores de R , T_1 y T_2 y léalos usando un script de Python. Realice tres mediciones de 20 segundos cada una en las que toque el sensor para provocar cambios en la temperatura, deje enfriar entre cada medición.
- i. Usando el código del sensor (B57164K0103) genere una tabla de valores de resistencia usando este [software](#), use $\Delta R/R$ de 5 % y 1°C de resolución, exporte los datos.

- j. Utilizando Python y los datos obtenidos en las tres mediciones calcule T_3 para cada valor de R usando interpolación lineal entre los puntos obtenidos por medio del software
- k. Para cada una de las mediciones realice una gráfica que muestre T_1 , T_2 y T_3

2.4. Análisis

- ¿Cual le parece que es el mejor método de aproximación? ¿Porqué?
- ¿Que tan importante es la precisión de la resistencia R_0 ?
- ¿Que tan importante es la precisión de la referencia del ADC?

Laboratorio 3

Sensores de presión

3.1. Objetivos

Al finalizar este laboratorio el estudiante estará en capacidad de:

- Utilizar un sensor para medir presión absoluta.

3.2. Materiales y equipo

- 1 Arduino UNO R4 MINIMA
- 1 sensor digital de presión BMP280
- 1 mini breadboard de 170 pines
- Jumpers macho-macho

3.3. Actividad 1

3.3.1. Procedimiento

- a. Conecte el sensor BMP280 tal como se muestra en la Figura 3.1
- b. Copie el siguiente código en el IDE de Arduino (el código se puede encontrar en: [GitHub](#))

```
#include "Adafruit_BMP280.h"

Adafruit_BMP280 bmp;
float temperature = 0;
float pressure = 0;

void setup() {
  Serial.begin(115200);
  delay(2000);
  if (!bmp.begin(0x77)) {
    Serial.println("Sensor not found");
    while (1) {}
  }
}
```

```

}

void loop() {
  // Read the values
  temperature = bmp.readTemperature();
  pressure = bmp.readPressure()/100; //hectopascales
  // Print to the Serial Monitor
  Serial.print(temperature);
  Serial.print(",");
  Serial.println(pressure);
  delay(1000);
}

```

- c. Analice cada línea de código y cuando entienda su funcionamiento corra el *sketch* usando el botón *Upload*
- d. Haga contacto con el encapsulado del sensor de forma que su dedos calienten el sensor, observe como cambia el valor en la terminal.
- e. Modifique el programa de forma que lo que se imprima en la terminal sea algo como esto:

```

24.8,806.1
24.7,806.3

```

- f. Instale las librerías *pyserial*, *matplotlib*, *drawnow* y *datetime* en Python para ser utilizadas luego.
- g. Copie el siguiente código en su IDE de Python. (el código se puede encontrar en: [GitHub](#))

```

# -*- coding: utf-8 -*-
import serial
import csv
from datetime import datetime
import matplotlib.pyplot as plt
from drawnow import drawnow

arduino = serial.Serial('COM14')
nombre = "bmp280"
narchivo = 1
mediciones = 10

def data_fig():
    plt.plot(sec, pres)

datos = []
sec = []
pres = []

arduino.write(b'0')

for i in range(mediciones):
    dato = arduino.readline()[:-2].decode('utf-8').split(',')
    print(dato)
    datos.append([datetime.now().strftime(format="%Y-%m-%d %H:%M:%S"),

```

```

        float(dato[0]),
        float(dato[1]))
    sec.append(i)
    pres.append(float(dato[1]))
    drawnow(data_fig)

seguir = input("Cualquier tecla")

# Guardar en CSV
with open(f"{nombre}_{narchivo}.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["timestamp", "Temperatura (C)", "Presion (hPa)"]) #
    encabezados
    writer.writerows(datos)

print("Datos guardados en mediciones.csv")

```

- h. Modifique el puerto COM para que coincida con el puerto en el que esta conectado el Arduino UNO R4 MINIMA
- i. Modifique el código para que cada toma de datos se almacene en un archivo *.csv* y para que se genere una gráfica en tiempo real.
- j. Realice tres tomas de datos de 60 segundos cada una

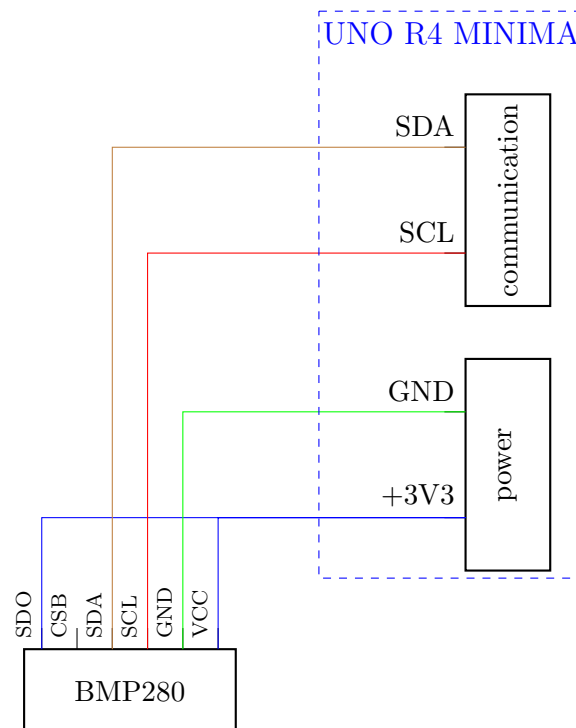


Figura 3.1: Conexión de sensor de presión absoluta BMP280

3.3.2. Análisis

- a. Determine la presión atmosférica local usando el promedio de las mediciones que las tres tomas de datos, incluya la precisión de la medición
- b. Determine la altitud local usando el valor de presión calculado ¿Es este valor cercano al esperado para la altitud del campus (1407 msnm)?

3.4. Actividad 2

3.4.1. Procedimiento

- a. Usando la misma configuración anterior, realice una nueva toma de datos de 60s en la que lleve el sensor lo mas cerca del piso posible y luego lo mas alto posible lentamente y repita durante los 60 segundos de la toma

3.4.2. Análisis

- a. ¿Se puede observar el cambio de altura en las mediciones?
- b. Según la hoja de datos del sensor ¿Debería detectar estos cambios?
- c. ¿El tiempo entre mediciones es realmente un segundo?

Laboratorio 4

Mediciones de nivel

4.1. Objetivos

Al finalizar este laboratorio el estudiante estará en capacidad de:

- Utilizar un sensor para medir nivel.
- Determinar la curva de calibración para medición de volumen.

4.2. Materiales y equipo

- 1 Arduino UNO R4 MINIMA
- 1 sensor ultrasónico HC-SR04
- 1 mini breadboard de 55 pines
- 1 beaker de 250 ml
- Jumpers macho-macho y hembra-hembra

4.3. Actividad 1

4.3.1. Procedimiento

- a. Conecte el sensor HC-SR04 tal como se muestra en la Figura 4.1
- b. Copie el siguiente código en el IDE de Arduino (el código se puede encontrar en: [GitHub](#))

```
/*  
 * Nivel  
 */  
  
const int trigPin = 9;  
const int echoPin = 10;  
  
float duration, distance, volume;
```

```
void setup() {  
  pinMode(trigPin, OUTPUT);  
  pinMode(echoPin, INPUT);  
  Serial.begin(9600);  
}  
  
void loop() {  
  digitalWrite(trigPin, LOW);  
  delayMicroseconds(2);  
  digitalWrite(trigPin, HIGH);  
  delayMicroseconds(10);  
  digitalWrite(trigPin, LOW);  
  
  duration = pulseIn(echoPin, HIGH);  
  distance = (duration*.0343)/2;  
  // distance = 7.96 - distance;  
  // volume = 32.05 * distance + 29.492;  
  Serial.print("H: ");  
  Serial.println(distance);  
  // Serial.print(" V:");  
  // Serial.println(volume);  
  delay(100);  
}
```

- c. Analice cada línea de código y cuando entienda su funcionamiento corra el *sketch* usando el botón *Upload*
- d. Fije el sensor en el aditamento plástico y coloquelo en la parte superior del beaker
- e. Realice una medición de distancia para poder calibrar el valor cero.
- f. calibre el sensor en cero de forma que pueda medir la altura desde la base el beaker
- g. Llene el beaker hasta 50ml, tome 10 datos y utilice el promedio para relacionar altura y volumen en ese punto
- h. Llene el beaker hasta 150ml, tome 10 datos y utilice el promedio para relacionar altura y volumen en ese punto
- i. Calcule la curva de calibración de volumen usando esos dos puntos
- j. Vacie el beaker
- k. Llene el beaker a 50, 100, 150 y 200ml y en cada uno de esos puntos tome el promedio de 10 datos de volumen como valor medido

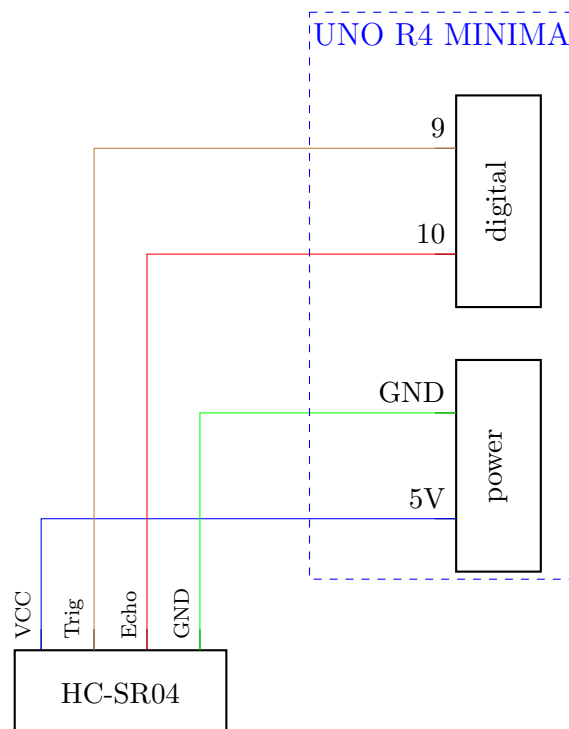


Figura 4.1: Conexión de sensor ultrasónico de distancia

4.3.2. Análisis

- Compare los valores medidos con los valores teóricos para 50, 100, 150 y 200ml
- Determine la incertidumbre por repetibilidad de la medición de volumen

4.4. Actividad 2

4.4.1. Procedimiento

- Usando la misma configuración anterior, realice una nueva toma de datos de 15 segundos en la que inicie con el beaker vacío y lo llene hasta aproximadamente 200ml

4.4.2. Análisis

- Incluya una gráfica del proceso de llenado
- ¿Se observar algún tipo de distorsión en la medición? Si es así, ¿a que se debe?

```
[12pt,letterpaper]report iftex fancyhdr lastpage amsmath physics amssymb siunitx float tikz url
[siunitx,american,RPvoltages]circuitikz tabularx [spanish,es-tabla,activeacute]babel csquotes [bac-
kend=biber,style=ieee]biblatex ../common/referencias.bib booktabs pgfplots hyperref bm bohr
elements verbatim [edges]forest etoolbox schemata appendix listings
color listings courier
csvsimple geometry left=18mm,right=18mm,top=21mm,bottom=21mm,headheight=15pt fancyhdr
```


Laboratorio 5

Sensores de peso

5.1. Objetivos

Al finalizar este laboratorio el estudiante estará en capacidad de:

- Implementar un sistema de medición de peso utilizando el módulo HX711 con calibración multipunto

5.2. Materiales y equipo

- 1 Arduino UNO R4 MINIMA
- Celda de carga de 1 kg
- 1 amplificador HX711
- Pesas de referencia (500g, 1000g, 1500g)
- Cable USB y computadora con Arduino IDE

5.3. Actividad 1

5.3.1. Procedimiento

- a. Conecte el amplificador HX711 tal como se muestra en la Figura
- b. Copie el siguiente código en el IDE de Arduino (el código se puede encontrar en: [GitHub](#))

```
#include "HX711.h"

HX711 scale;

#define DATAPIN 6
#define CLOCKPIN 7

void setup()
{
    Serial.begin(115200);
```

```

    delay(1000);

    scale.begin(DATAPIN, CLOCKPIN);
}

String input = "a";
String maxt = "500";
int maxn = 500;
float value = 0.0;
float units = 0.0;

void loop()
{
    value = scale.get_value(2);
    units = scale.get_units(10);
    Serial.print(value);
    Serial.print(",");
    Serial.println(units);
    delay(250);
    while(Serial.available()) input = Serial.readStringUntil('\n');
    if (input == "c") {
        Serial.println("***Calibration***");
        Serial.println("\nEmpty the scale, press a key to continue");
        while(!Serial.available());
        while(Serial.available()) Serial.read();
        scale.tare();
        Serial.print("Weight: ");
        Serial.println(scale.get_units(10));
        Serial.println("\nSet max weight");
        while(!Serial.available());
        while(Serial.available()) maxt = Serial.readStringUntil('\n');
        maxn = maxt.toInt();
        Serial.print("\nPut ");
        Serial.print(maxn);
        Serial.println(" gram in the scale, press a key to continue");
        while(!Serial.available());
        while(Serial.available()) Serial.read();
        scale.calibrate_scale(maxn, 10);
        Serial.print("Weight: ");
        Serial.println(scale.get_units(10));
        input = "n";
    }
}

```

- c. Analice cada línea de código y cuando entienda su funcionamiento corra el *sketch* usando el botón *Upload*
- d. En la terminal serial ingrese la letra *c*, esto iniciará un proceso de calibración, siga las instrucciones del menú, calibre con peso máximo de 200 gramos.
- e. Copie el siguiente código en su IDE de Python. (el código se puede encontrar en: [GitHub](#))

```

import serial
import csv

```

```
from datetime import datetime

arduino = serial.Serial('COM9')
nombre = "scale900_800"
narchivo = 3
mediciones = 20

datos = []

arduino.write(b'0')

for i in range(mediciones):
    dato = arduino.readline()[:-2].decode('utf-8').split(',')
    print(dato)
    datos.append([datetime.now().strftime(format="%Y-%m-%d %H:%M:%S"),
                  float(dato[0]),
                  float(dato[1])
                  ])

# Guardar en CSV
with open(f"{nombre}_{narchivo}.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["timestamp", "raw", "peso"]) # encabezados
    writer.writerows(datos)

print("Datos guardados en mediciones.csv")
```

- f. Modifique el puerto COM para que coincida con el puerto en el que esta conectado el Arduino UNO R4 MINIMA.
- g. Coloque un peso de 100 gramos y realice una toma de datos de 20 muestras.
- h. Coloque un peso de 300 gramos y realice una toma de datos de 20 muestras.
- i. Repita este procedimiento para calibraciones con peso máximo de 500 gramos y 900 gramos.

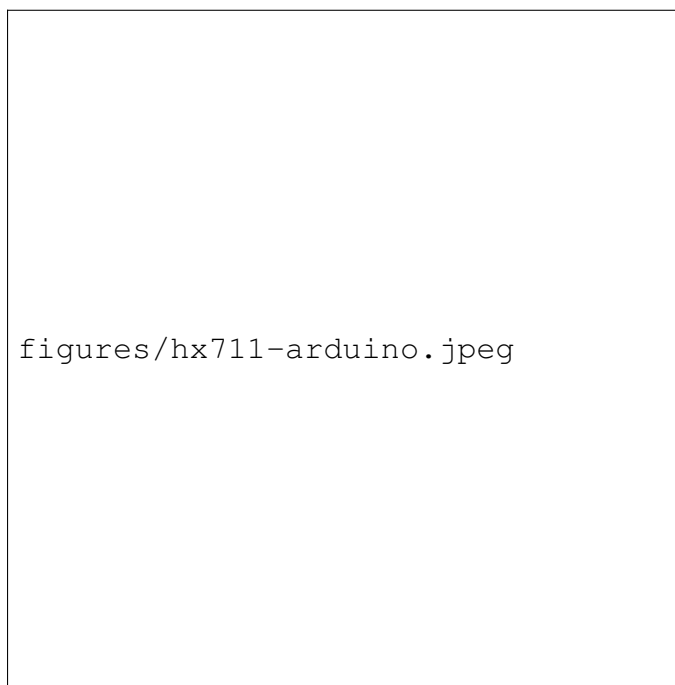


Figura 5.1: Proceso de calibración del módulo HX711

5.3.2. Análisis

- a. ¿Cuál es la resolución teórica del módulo HX711 si trabaja con un convertidor de 24 bits y una celda de carga de 1 kg? ¿Qué implicaciones tiene esto en la medición de masas pequeñas? Como recomendación, utilizar la siguiente ecuación:

$$\text{Resolución} = \frac{\text{Capacidad de la celda}}{2^{\text{Número de bits}}}$$

- b. ¿Observó alguna diferencia entre las mediciones realizadas con diferentes puntos de calibración? ¿Cuál fue mejor?